

PROJECT REPORT

ON

TIC TAC TOE

Submitted in partial fulfilment of the requirements for the award of degree
of

Bachelor of Computer Application (BCA)

To

UTTRANCHAL SCHOOL OF COMPUTING SCIENCES

UTTRANCHAL UNIVERSITY, DEHRADUN



Under the guidance of

Ms. Divya Singh

(Assistant Professor)

Submitted By

Anshika Choudhary

Roll no. 08

BCA-1B

(Batch: 2024-2027)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to Director, Prof. (Dr.) Sonal Sharma, my project supervisor, Ms. Divya Singh for the valuable guidance, insights, and encouragement. Their expertise has been instrumental in shaping this project. I feel honored to be a part of this project for which I put all my heart into and shown utmost sincerity to complete it. The resources provided by our college were valuable as I was able to enhance my knowledge and got to learn something new. I also want to acknowledge my peers and friends for their constant support and constructive feedbacks which helped me refine my ideas and approach. Finally, I appreciate my family for their unwavering encouragement and understanding during this process.

Thank you all for being an essential part of this journey.

DECLARATION

I, Anshika Choudhary , hereby declare that the project work entitled "Tic Tac Toe" submitted for the partial fulfillment of BCA is my original work. I have carried out this project work independently under the guidance of Miss Divya Singh , and it has not been submitted to any other institution or organization for the award of any degree or diploma.

I further declare that the information derived from various sources has been duly acknowledged in the report, and no part of this project has been plagiarized or copied without proper citation.

Date: 30-09-2024

Signature: _____

Anshika Choudhary

Roll no. 08

Student ID 24360900075

CERTIFICATE

This is to certify that the project report titled "Tic Tac Toe" submitted by Anshika Choudhary , Roll no. 08 , in partial fulfillment of the requirements for the degree of BCA, is a record of original work carried out by him/her/them under my supervision and guidance.

The project work was carried out during the academic year 1st and it has not been submitted previously to any other institution or university for the award of any degree or diploma.

Date: 30-09-2024

Table of Contents

S.No	Topic	Page No
1	Acknowledgement	2
2	Declaration	3
3	Certificate of Originality	4
4	Table of Contents/Index with page numbering	5
5	Introduction	6
6	Objective	7
7	Tools & Platform	8
8	Hardware And Software Requirement	10
9	System Analysis	12
10	Identification Of Need	16
11	Preliminary Investigation	18
12	Fesibility Study	21
13	Project Planning	24
14	Software Requirement Specification	26
15	Software Engineering Paradigm Applied	30
16	Control Diagram	32
17	System Design	33
18	Modularisation Details	36
19	User Interface Design	37
20	Testing	45
21	Testing Technique And Testing Strategies	47
22	System Security	50
23	Future Scope And Future Enhancement Of The Project	53
24	Appendices 1. Coding	57
25	2. Bibliography	70

Introduction:-

Introduction to Tic Tac Toe

Tic Tac Toe, also known as "Noughts and Crosses" or "X's and O's," is a simple yet classic two-player game. It is one of the most well-known games for developing strategic thinking and logical reasoning, especially among children. The game is easy to learn, quick to play, and requires no special equipment, making it a favorite for players of all ages.

- **Players:** Two participants take turns. One is assigned "X," and the other is "O."
- **Game Board:** A grid of 3 rows and 3 columns (a total of 9 spaces).
- **Objective:** The goal is to align three of your symbols (X or O) in a row, column, or diagonal while preventing your opponent from doing the same.

Rules

1. The players alternate turns, placing their symbol (X or O) in an empty space on the grid.
2. A player wins if they successfully form a straight line of three symbols, either horizontally, vertically, or diagonally.
3. If all spaces are filled without a winning line, the game ends in a draw or tie.

Despite its apparent simplicity, Tic Tac Toe offers a surprising depth of strategy:

- Players must anticipate their opponent's moves while planning their own.
- It teaches foresight, as players need to think multiple steps ahead.
- The game often ends in a draw when both players are experienced, demonstrating the importance of tactical defense.

Variants and Extensions

While the standard 3x3 grid is the most common, larger grids (like 4x4 or 5x5) and rule modifications add complexity. Variants such as "3D Tic Tac Toe" or "Ultimate Tic Tac Toe" offer fresh challenges and extended strategic depth.

Tic Tac Toe remains a timeless example of how simplicity and strategy can merge into an engaging and educational pastime.

Objective:-

The primary objective of Tic Tac Toe is to outsmart your opponent by being the first to form a winning line of three identical symbols (X or O) on the grid. This line can be achieved in one of the following ways:

1. **Horizontally:** Placing three symbols in a straight row.
2. **Vertically:** Placing three symbols in a straight column.
3. **Diagonally:** Placing three symbols in a diagonal line from one corner to the opposite corner.

Secondary Objectives

1. **Preventing the Opponent from Winning:** Block your opponent's moves to stop them from forming a winning line.
2. **Strategic Thinking:** Make moves that set up multiple potential wins, forcing your opponent into a defensive position.
3. **Achieving a Draw:** If winning is not possible, aim to block all potential winning moves by your opponent to end the game in a tie.

The game encourages logical thinking, planning ahead, and learning from mistakes, making it more than just a pastime—it is a valuable exercise in strategy and problem-solving.

Tools & Platform:-

Tic Tac Toe is a versatile game that can be played using various tools and on different platforms, ranging from traditional physical setups to advanced digital formats. Here's an overview:

1. Physical Tools

- **Paper and Pencil:**
The simplest way to play. Players draw a 3x3 grid on a piece of paper and mark X's and O's. It's portable, cost-effective, and requires no technology.
- **Chalkboards/Whiteboards:**
Popular in classrooms or group settings, where players use chalk or markers to draw and play the game.
- **Game Boards:**
Pre-made Tic Tac Toe boards with reusable pieces (X's and O's) are often available as physical games for home or travel.

2. Digital Platforms

- **Mobile Apps:**
Many Tic Tac Toe apps are available on Android and iOS platforms, offering features like AI opponents, multiplayer modes, and custom game variations. Examples include *Tic Tac Toe Glow* and *Ultimate Tic Tac Toe*.
- **Computer Programs:**
Playable on personal computers through simple applications or web-based versions. Some programs include customizable difficulty levels and engaging user interfaces.
- **Web Browsers:**
Numerous online websites host free versions of Tic Tac Toe, allowing players to compete against the computer or other online users. Examples include:
 - **Playtictactoe.org**
 - **Math Playground**
- **Gaming Consoles:**
Some consoles feature Tic Tac Toe as a mini-game or within puzzle collections.

3. Coding Platforms

- **Programming Languages:**
Enthusiasts often use platforms like C to build their own versions of Tic Tac Toe. This is a popular beginner programming project.
- **Game Development Engines:**
Platforms like Unity or Scratch are used to create visually appealing or more complex versions of the game, such as 3D Tic Tac Toe.

4. Social Platforms

- **Multiplayer Games:**
Tic Tac Toe is often included as a feature in social platforms and messaging apps like Facebook Messenger, allowing friends to play together in real-time.
- **Physical Toy Platforms:**
Electronic versions of Tic Tac Toe in toy form are available, with light-up grids and automated scoring systems for solo or multiplayer play.

5. Educational Platforms

Tic Tac Toe is frequently integrated into educational tools to teach logic, critical thinking, and strategic planning. Platforms such as Khan Academy or custom classroom apps may include it as a learning activity.

By leveraging these tools and platforms, players can enjoy Tic Tac Toe in a variety of formats, from casual play to competitive or educational settings.

HARDWARE AND SOFTWARE REQUIREMENTS-

1. Physical Format Requirements

Hardware

- **Paper** and **Pencil:**
A simple paper and pencil/pen for drawing the 3x3 grid and marking moves.
- **Pre-made Game Boards:**
 - o A physical Tic Tac Toe board (e.g., wood, plastic, or cardboard).
 - o Reusable pieces (X's and O's) or markers.

2. Digital Format Requirements

Hardware

1. **For Mobile Devices:**
 - o A smartphone or tablet with:
 - Minimum specifications:
 - 1GB RAM or higher (for simple apps).
 - iOS/Android operating system.
 - Screen resolution of at least 720p for a clear display.
 - o Touchscreen functionality for ease of interaction.
2. **For Computers:**
 - o A personal computer (desktop or laptop) with:
 - Minimum 1GB RAM.
 - Dual-core processor (sufficient for lightweight games).
 - Basic GPU or integrated graphics for 2D games.
 - Monitor with standard resolution (720p or higher).
 - o Keyboard and mouse for navigation.
3. **For Gaming Consoles:**
 - o A gaming console such as a Nintendo Switch, PlayStation, or Xbox with pre-installed or downloadable Tic Tac Toe games.
 - o Controllers for interaction.
4. **For Interactive Boards (Classroom Use):**
 - o A digital whiteboard or smartboard with stylus input capability.

Software

1. Mobile Devices:

o Operating System:

- Android 5.0 or later.
- iOS 9.0 or later.
- o Tic Tac Toe apps available on app stores, such as *Tic Tac Toe Glow* or *Classic Tic Tac Toe*.

2. Web Browsers (Cross-Platform):

- o A modern web browser like Google Chrome, Mozilla Firefox, or Safari.
- o Stable internet connection for accessing online Tic Tac Toe platforms.

3. For Programmers and Coders:

o Programming IDEs/Tools:

- Python: Use IDLE or PyCharm.
- JavaScript: Use VS Code or any text editor with a browser for running scripts.
- Scratch: Use its official web platform or desktop app for creating a visual Tic Tac Toe game.
- o Lightweight engines like Unity (for advanced versions).

4. PC Games:

- o Lightweight software or browser-based games compatible with Windows, macOS, or Linux.

Recommended Configurations

For casual play:

- A basic smartphone or PC will suffice for most Tic Tac Toe games.

For creating or experimenting with custom games:

- A computer with coding tools installed (e.g., Python or Unity).

Tic Tac Toe is a minimal-requirement game, making it accessible on nearly all modern devices.

System Analysis

1. System Overview

Tic Tac Toe is a turn-based, two-player game played on a 3x3 grid. Players alternate turns, aiming to align three symbols (X or O) in a row, column, or diagonal. The system includes basic input handling, rule enforcement, and win/draw detection.

2. Functional Requirements

These are the core functionalities the system should perform:

Inputs:

- Allow players to select and place their symbol (X or O) on the grid.
- Validate that the chosen cell is empty before accepting the move.

Game Logic:

- Switch turns between Player 1 and Player 2.
- Check after each move for:
 1. A winning condition (three symbols aligned).
 2. A draw condition (all cells filled with no winner).

Outputs:

- Display the current state of the grid after each move.
- Indicate the result:
 - o Win (e.g., "Player 1 Wins!")
 - o Draw (e.g., "It's a tie!")
- Allow a replay option or exit.

3. Non-Functional Requirements

These are the quality attributes of the system:

- **Usability:** The interface should be simple and intuitive for all age groups.
- **Performance:** Minimal processing time for each move, ensuring seamless play.
- **Reliability:** Accurately detect wins, draws, and enforce rules.
- **Portability:** The system should work across multiple platforms (e.g., web, mobile, desktop).
- **Scalability** (optional): Extendable to larger grids or variants like Ultimate Tic Tac Toe.

4. Technical Requirements

Hardware Requirements:

- Device with basic input capabilities (keyboard, touchscreen, or mouse).
- Display device for the 3x3 grid interface.

Software Requirements:

- **For Manual Play:** No software needed (paper/board suffices).
- **For Digital Play:**
 - o A programming language or platform (e.g., Python, JavaScript, Unity).
 - o A graphical user interface (GUI) for enhanced user interaction (optional).

5. System Components

Game Board:

- A 3x3 matrix (array or list) to store the state of the game (e.g., empty, X, O).
- Methods to update and display the board.

Players:

- Two entities (Player 1 and Player 2).
- Each player has a unique identifier (X or O).

Rules Engine:

- Validates moves (e.g., ensures a chosen cell is empty).
- Determines game outcomes (win/draw).

User Interface (UI):

- Displays the game board and provides input mechanisms (e.g., mouse clicks, touch, or typing coordinates).

Control Logic:

- Handles the sequence of turns.
- Invokes the rules engine after each move.

6. System Flow

Input-Process-Output (IPO) Model:

1. **Input:**
 - o Player selects a cell on the grid.
2. **Process:**
 - o Validate the move.
 - o Update the board.
 - o Check for win or draw conditions.
3. **Output:**
 - o Display the updated board and results.

7. Constraints

- The game must limit moves to 9 turns.
- Each move must be unique and occupy an unclaimed cell.
- Players must alternate turns.

8. Feasibility Analysis

Operational Feasibility:

- Simple for players to understand and operate, requiring minimal instructions.

Technical Feasibility:

- Minimal hardware/software resources required.
- Can be implemented with basic programming knowledge.

Economic Feasibility:

- Low cost for manual play (paper and pencil) or digital play (basic development tools are free).

9. Conclusion

The system for Tic Tac Toe is straightforward to analyze and implement, making it an excellent project for beginners in game development or system design. It's versatile and adaptable for additional features or rule modifications, allowing for scalability and creative customization.

Identification of need

The need for creating and playing Tic Tac Toe arises from various educational, recreational, and developmental purposes. Below are the primary reasons and situations where Tic Tac Toe fulfills a specific need:

1. Recreational Need

- **Entertainment:** Provides a quick and engaging activity for leisure time, suitable for players of all ages.
- **Accessibility:** Easy to set up and play anywhere with minimal tools (paper/pencil or a digital device).
- **Low-cost Fun:** Requires little to no expense, making it accessible to everyone.

2. Educational Need

- **Strategic Thinking:** Encourages players to think ahead and anticipate their opponent's moves.
- **Problem-Solving Skills:** Teaches logical reasoning and decision-making under constraints.
- **Pattern Recognition:** Helps players identify winning combinations and block opponents effectively.
- **Teaching Tool for Children:** Often used as an introduction to games that involve rules, competition, and strategy.

3. Developmental Need

- **Cognitive Skills:** Enhances memory, focus, and planning abilities.
- **Social Interaction:** Fosters healthy competition and communication between players, improving social skills.
- **Emotional Development:** Teaches the concepts of winning, losing, and resilience.

4. Technological or Creative Need

- **Programming and Development Practice:**
 - o A great project for beginners in coding, as it involves fundamental concepts like loops, conditions, and arrays.
 - o Provides a platform to experiment with user interfaces, artificial intelligence, and game logic.
- **Customization:** Developers can modify the game to include new rules, variations, or advanced designs, catering to creative and experimental needs.

5. Practical Need

- **Time Efficiency:** A quick game that can be completed in minutes, ideal for filling short time gaps.
- **Portability:** Can be played virtually anywhere, from classrooms to online platforms.

6. Social and Cultural Need

- **Universal Appeal:** Known worldwide, making it a common game that bridges cultural and language barriers.
- **Icebreaker Activity:** Serves as a fun and lighthearted way to connect with others in social or educational settings.

Conclusion

The need for Tic Tac Toe lies in its simplicity, versatility, and the wide range of benefits it offers, from entertainment to skill development. Whether as a pastime, a teaching tool, or a programming exercise, Tic Tac Toe remains a relevant and valuable activity in various contexts.

Preliminary Investigation

A preliminary investigation is the initial step in analyzing and understanding the scope, feasibility, and requirements of a project or system. For Tic Tac Toe, whether manual or digital, this phase identifies objectives, assesses needs, and evaluates possible approaches.

1. Objective of the Investigation

- Determine the purpose of developing or implementing Tic Tac Toe.
- Identify stakeholders (e.g., players, developers, educators).
- Assess the feasibility of creating a physical or digital version of the game.
- Outline the benefits, limitations, and potential extensions of the system.

2. Problem Statement

Manual Version:

- Simple to play but lacks automation (e.g., rule enforcement or scoring).
- Requires human moderation to detect wins or draw conditions.

Digital Version:

- More engaging and accessible but requires clear objectives and resources for implementation.
- Players may seek additional features like AI opponents, difficulty levels, or multiplayer support.

3. Investigation Steps

Feasibility Study

1. **Technical Feasibility:**
 - o Evaluate tools required for development (e.g., programming languages, platforms like Python, Unity, or Scratch).
 - o Assess hardware requirements (e.g., PC, smartphone, or console).
2. **Operational Feasibility:**
 - o Check if the intended audience can easily understand and use the game.
 - o Ensure user-friendly design for both players and developers.
3. **Economic Feasibility:**
 - o Minimal cost for physical formats (paper/pencil or game boards).
 - o For digital formats, determine development and maintenance costs.

Requirements Analysis

1. **Functional Requirements:**
 - o Player input mechanisms (manual or digital).
 - o Automated win/draw detection for digital versions.
 - o Turn-based system and display updates.
2. **Non-Functional Requirements:**
 - o User-friendly interface for digital versions.
 - o Real-time responsiveness.
 - o Cross-platform compatibility.
3. **Optional Features:**
 - o Single-player mode with AI.
 - o Customizable board sizes or rule variations.
 - o Multiplayer functionality (local or online).

Research and Benchmarking

- Examine existing Tic Tac Toe implementations (physical and digital).
- Identify gaps or improvements (e.g., AI sophistication, graphical enhancements).
- Consider extensions like "Ultimate Tic Tac Toe" or 3D Tic Tac Toe for future development.

4. Outcomes of the Investigation

For Manual Versions:

- Feasible to implement immediately with basic materials.
- Effective for teaching strategy and logic without technology.

For Digital Versions:

- Requires basic programming and design skills.
- Feasible for mobile, web, and desktop platforms.
- Can include additional features for enhanced gameplay and user experience.

5. Recommendations

- Proceed with digital development if automation, AI, or multiplayer features are desired.
- Choose an appropriate platform (e.g., web for accessibility, mobile for portability).
- Start with a simple version and incrementally add features like difficulty levels or customizations.

Conclusion

The preliminary investigation confirms that Tic Tac Toe is feasible to implement in both manual and digital formats. It meets the needs of entertainment, education, and skill development with minimal resources, making it a worthwhile project for various contexts.

Feasibility Study

A feasibility study evaluates whether the development and implementation of a Tic Tac Toe game are practical and achievable. This involves analyzing technical, operational, economic, legal, and scheduling aspects.

1. Types of Feasibility

Technical Feasibility

- **Requirements:**
 - **Manual Version:** Paper, pencil, or a physical board.
 - **Digital Version:**
 - **Hardware:** Computer, smartphone, or gaming console.
 - **Software:** Programming languages (Python, JavaScript), game engines (Unity, Scratch), or GUI libraries (Tkinter, React).
 - **AI Implementation** (if needed): Algorithms like Minimax for AI decision-making.
- **Assessment:**
 - Easy to implement on modern devices.
 - Suitable for basic hardware and lightweight software.
 - No specialized tools required for standard versions.
- **Conclusion:** Highly feasible technically.

Operational Feasibility

- **Target Audience:**
 - Casual players, students, educators, or developers learning programming.
 - Suitable for all age groups and skill levels.
- **Ease of Use:**
 - Intuitive rules and user-friendly interface ensure accessibility.
 - Online or app-based versions make the game widely available.
- **Maintenance:**
 - Low maintenance for physical versions.
 - Digital versions require occasional updates for bug fixes or feature additions.
- **Conclusion:** Operationally feasible, with no significant barriers to user adoption.

Economic Feasibility

- **Manual Version:**
 - Low-cost materials (paper/pencil or pre-made boards).
 - No recurring expenses.
- **Digital Version:**
 - Development costs: Minimal for a basic version (free tools like Python or Scratch).
 - Distribution: Free platforms (e.g., Google Play, web hosting) can be used for accessibility.

- o Scalability: Optional advanced features (AI or customizations) may increase development time but are cost-effective for personal or educational purposes.
- **Conclusion:** Economically feasible with low cost and high ROI in entertainment or educational value.

Legal Feasibility

- **Intellectual Property:**
 - o Tic Tac Toe is a public domain game with no copyright issues.
 - o Custom designs, branding, or advanced versions may need trademark considerations.
- **Platform Compliance:**
 - o Digital versions must adhere to the guidelines of app stores or web hosting platforms.
- **Conclusion:** Legally feasible with minimal risks.

E. Scheduling Feasibility

- **Development Timeline:**
 - o Manual Version: Immediate setup and play.
 - o Basic Digital Version:
 - Simple functionality: 2–5 days for experienced developers.
 - Advanced features: 1–2 weeks, depending on the complexity (e.g., AI or multiplayer).
- **Testing and Deployment:**
 - o Testing for functionality and bug fixes can be completed in 1–2 days.
 - o Deployment via web or app stores is straightforward.
- **Conclusion:** Feasible within a short development cycle.

2. Risk Assessment

- **Technical Risks:** Limited for manual versions. For digital, potential bugs in AI or user interface.
- **Operational Risks:** Low engagement if features are too basic.
- **Economic Risks:** Minimal for non-commercial versions.

3. Recommendations

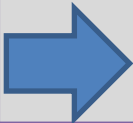
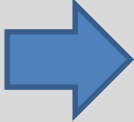
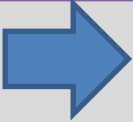
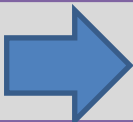
- Start with a basic digital or manual version to minimize costs and risks.
- Gradually introduce advanced features, like AI or multiplayer, to enhance engagement.
- Choose free development tools and platforms to reduce expenses.

4. Conclusion

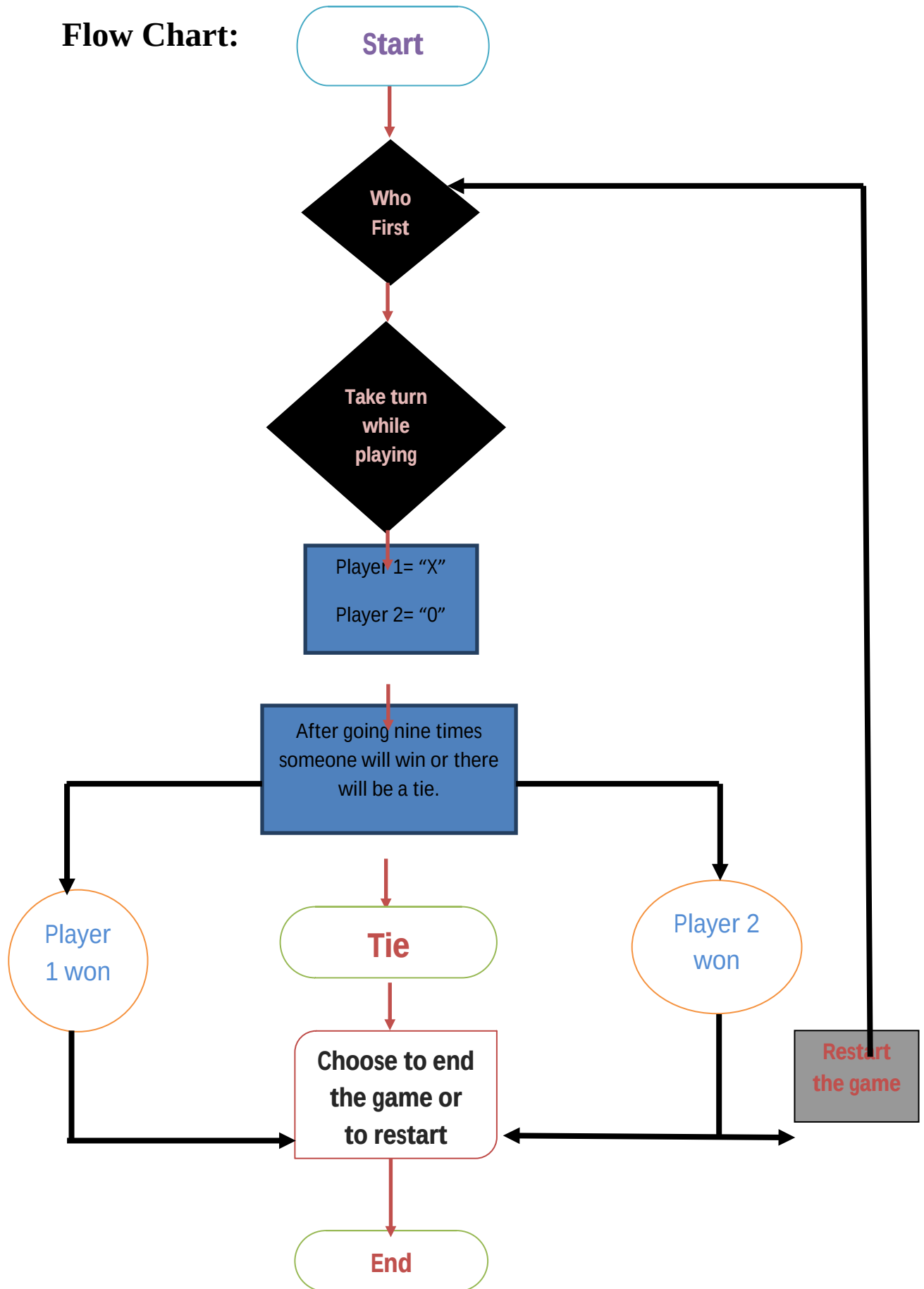
The feasibility study indicates that developing Tic Tac Toe, whether as a manual or digital version, is highly achievable and practical. The low costs, minimal technical challenges, and broad appeal make it an excellent project for personal, educational, or recreational purposes.

Project Planning

Gantt Chart:

WEEK	Week1	Week2	Week3	Week4	Week5	Week6
TASK						
Project introduction & objective						
Game mechanism						
Ghost AI & collision detection						
Scoring system, lives, & power-ups						
Testing, debugging						
Final polishing & deployment						

Flow Chart:



Software Requirement Specification

1. Introduction

This document outlines the functional and non-functional requirements for the development of a Tic-Tac-Toe game.

1.1 Purpose

The purpose of this document is to define the requirements for a simple, interactive Tic-Tac-Toe game for entertainment purposes. The game will allow single-player and multiplayer modes with a user-friendly interface.

1.2 Scope

The Tic-Tac-Toe game will be implemented as a standalone application, accessible via a web browser, desktop, or mobile platform. Users can either play against another user (multiplayer) or the computer (single-player with AI). The game will follow the classic 3x3 grid rules.

1.3 Definitions, Acronyms, and Abbreviations

- **Tic-Tac-Toe:** A game played on a 3x3 grid where two players alternately place marks (X or O) with the goal of aligning three marks in a row, column, or diagonal.
- **AI:** Artificial Intelligence.
- **GUI:** Graphical User Interface.

1.4 References

- Tic-Tac-Toe rules: [Wikipedia](#)
- Design patterns for games: "Game Programming Patterns" by Robert Nystrom

2. Overall Description

2.1 Product Perspective

This game is a new application intended for casual gaming. It uses a simple grid layout for gameplay and standard game mechanics.

2.2 Product Features

- Two modes:
 - o **Single-player:** Player vs. computer (with basic AI).
 - o **Multiplayer:** Player vs. Player (local).
- Visual 3x3 grid interface with intuitive controls.
- Score tracking system.
- Restart and exit options.

2.3 User Classes and Characteristics

- **Casual Players:** Require an easy-to-understand interface.
- **Competitors:** May seek a challenging AI opponent.

2.4 Operating Environment

- Platforms:
 - o Web browsers (Chrome, Firefox, Edge).
 - o Windows, macOS, Android, iOS.
- Minimum hardware requirements:
 - o 1 GHz processor.
 - o 512 MB RAM.
 - o 10 MB of free storage.

2.5 Constraints

- The game must adhere to the classic Tic-Tac-Toe rules.
- AI difficulty will be limited to basic strategies.

2.6 Assumptions and Dependencies

- The user has basic familiarity with Tic-Tac-Toe rules.
- The platform's graphical libraries and APIs are accessible.

3. Specific Requirements

3.1 Functional Requirements

- **FR1:** The system must display a 3x3 grid at the start of the game.
- **FR2:** Players must alternate turns.
- **FR3:** The system must detect and declare the winner or a draw.
- **FR4:** In single-player mode, the system must implement AI to make moves.
- **FR5:** Players must be able to restart the game or exit at any time.

3.2 Non-Functional Requirements

- **NFR1:** The system must provide a response within 1 second of a player's action.
- **NFR2:** The application should use minimal resources to ensure compatibility with low-end devices.
- **NFR3:** The game interface must be visually appealing and intuitive.

3.3 Performance Requirements

- Must support at least two users simultaneously in multiplayer mode.
- Must load within 3 seconds on supported platforms.

3.4 Security Requirements

- No personal data collection is required.
- Local storage of scores and settings must be secure.

3.5 User Interface Requirements

- A clean, minimalist grid layout.
- Buttons for restart and exit.
- Clear visual indicators for player turns and game outcomes.

4. Appendices

- **A1:** Example AI logic: Minimax algorithm.
- **A2:** Future Enhancements: Online multiplayer mode, custom grid sizes.

This SRS ensures a well-defined scope and structured development for the Tic-Tac-Toe game.

Software Engineering Paradigm Applied

The choice of a software engineering paradigm depends on the scope, complexity, and goals of the Tic-Tac-Toe project. For this simple project, **Incremental Development** and **Object-Oriented Paradigm** are suitable choices.

1. Incremental Development Paradigm

This paradigm involves developing the application in small, manageable increments, where each increment adds a functional module to the system.

Why it fits Tic-Tac-Toe:

- The game is relatively small in scope and can be divided into distinct features (e.g., grid design, player input, AI, result detection).
- Developers can focus on implementing and testing one feature at a time, reducing complexity.
- Allows continuous feedback and iterative refinement.

Steps in Incremental Development:

1. **Plan the basic system:** Implement the 3x3 grid and basic player interaction.
2. **Add single-player mode:** Integrate AI functionality.
3. **Enhance UI:** Include visual enhancements like player turn indicators and animations.
4. **Implement score tracking:** Add functionality to display and store scores.
5. **Finalize with testing:** Test the game for bugs and performance issues.

2. Object-Oriented Paradigm (OOP)

Tic-Tac-Toe can benefit from OOP due to its modular and reusable nature.

Key Concepts Applied to Tic-Tac-Toe:

1. **Encapsulation:**
 - o Each element of the game (e.g., grid, players, AI) can be represented as a class with its own attributes and methods.
 - o Hides implementation details, allowing changes without affecting other components.
2. **Abstraction:**
 - o Define high-level methods such as `makeMove()`, `checkWinner()`, and `restartGame()` that handle complex logic internally.
3. **Inheritance:**
 - o The Player class can have subclasses for `HumanPlayer` and `AIPlayer`, sharing common functionality while enabling specific implementations.
4. **Polymorphism:**
 - o Methods like `makeMove()` can behave differently depending on whether the player is human or AI.

Example Classes:

- **Grid:** Manages the 3x3 grid, validates moves, and checks for winners.
- **Player:** Base class for player interactions.
- **AIPlayer:** Inherits from Player and implements decision-making logic.
- **GameController:** Handles game flow, turn switching, and restarting.

Paradigm Application Summary

The **Incremental Development Paradigm** ensures a step-by-step and test-driven approach, ideal for small games like Tic-Tac-Toe. Meanwhile, the **Object-Oriented Paradigm** makes the code modular, maintainable, and extensible for future enhancements (e.g., adding larger grids, network multiplayer). Both paradigms together provide a structured and scalable development methodology.

Control Flow Diagrams

A **Control Flow Diagram** for a Tic Tac Toe game visually represents the decision-making and logical flow in the game's operation. Here's a breakdown of what such a diagram might include:

Key Steps in the Control Flow Diagram

1. **Start Game**
 - o Initialize the game board.
 - o Decide which player goes first.
2. **Player Turn**
 - o Display the current state of the board.
 - o Prompt the current player to make a move.
 - o Validate the move (check if it's a valid and empty cell).
3. **Update Board**
 - o Place the player's marker (X or O) on the board.
 - o Check for a winning condition:
 - If a player wins, display the winner and end the game.
 - o Check for a draw:
 - If the board is full and no player has won, declare a draw.
4. **Switch Turn**
 - o Change the active player to the next one.
5. **Repeat Until End**
 - o Repeat steps 2–4 until there is a winner or a draw.
6. **End Game**
 - o Display the final result.
 - o Option to restart or exit the game.

System Design

Designing a **Tic Tac Toe** system involves creating a framework for players to play the game while ensuring fairness, simplicity, and ease of use. Here's a breakdown of the system design:

1. Requirements

Functional Requirements:

- Players should be able to take turns making moves.
- Validate moves (e.g., ensure a spot is not already taken).
- Detect the game outcome:
 - Win: Three marks in a row (horizontally, vertically, or diagonally).
 - Draw: No more available moves and no winner.
 - Ongoing: Game is still in progress.
- Provide a restart option after a game ends.

Non-Functional Requirements:

- Low latency in processing moves.
- Scalability for multiple game sessions (if multiplayer or online).

2. System Components

Frontend (User Interface):

- **Platform:** Mobile app, web app, or desktop GUI.
- **UI Elements:**
 - 3x3 grid for the board.
 - Player indicators (whose turn it is).
 - Buttons for actions like "Restart" or "Undo" (optional).

Backend (Logic & State Management):

- **Core Modules:**
 1. **Game State Manager:**
 - Tracks the board state (e.g., 3x3 array).
 - Stores player turns and validates moves.
 2. **Move Validator:**
 - Ensures moves are within bounds and the cell is not already occupied.
 3. **Win Checker:**
 - Checks for a winning condition after every move.
 - Detects draw state.
 4. **Session Manager:**
 - Handles multiple games if multiplayer.

- **APIs (if multiplayer):**
 - o POST /move: Accepts the player's move.
 - o GET /state: Returns the current game state.
 - o POST /restart: Resets the board.

Database (Optional for Multiplayer or Persisted Games):

- **Schema Example:**
 - o GameSession: Game ID, players, board state, game status.
 - o Player: Player ID, name, score.

4. System Flow

1. **Game Initialization:**
 - o Create a new board.
 - o Assign players X and O.
2. **Player Makes a Move:**
 - o Frontend sends the move to the backend.
 - o Backend validates and updates the board.
 - o Backend checks for game outcome (win/draw/ongoing).
3. **Game State Update:**
 - o Frontend fetches and displays the updated board and game status.
4. **End Game:**
 - o Declare the winner or a draw.
 - o Provide the option to restart or quit.

5. Technologies

- **Frontend:** React (Web), Flutter (Mobile), or any simple UI library.
- **Backend:** Python (Flask/Django), Node.js, or Java (Spring).
- **Database:** SQLite or NoSQL (e.g., MongoDB) for session management.
- **Hosting:** AWS, Heroku, or Firebase for multiplayer games.

6. Scalability

For a multiplayer version:

- Use WebSockets or real-time frameworks (e.g., Socket.IO) for live updates.
- Store game states in a centralized database.
- Implement a matchmaking system for pairing players.

7. Advanced Features (Optional)

- **AI Opponent:** Implement a minimax algorithm for single-player games.
- **Undo Feature:** Maintain a move history stack.
- **Analytics:** Track player statistics like win rate.

Modularisation Details

1. Modules Overview

Modules in the Tic Tac Toe Design:

1. **Initialization Module** - Sets up the game board and initializes players.
2. **Game State Module** - Manages the game state (board and turn).
3. **Move Validator Module** - Validates player moves.
4. **Win Checker Module** - Checks for a win, draw, or ongoing game.
5. **Game Controller Module** - Orchestrates the game flow.
6. **Display Module** - Handles the UI for displaying the board and game status.
7. **Input Module** - Collects input from the players.
8. **Restart/Session Management Module** - Allows restarting the game.

Module Implementation in C

1. Initialization Module
2. Game State Module
3. Move Validator Module
4. Win Checker Module
5. Game Controller Module
6. Display Module
7. Input Module
8. Restart/Session Management Module

How This Modularization Helps

1. **Code Clarity:** Each module handles a distinct functionality, making the code easier to read.
2. **Reusability:** Modules like Win Checker and Move Validator can be reused in other games or versions.
3. **Testability:** Modules can be tested individually for bugs.
4. **Scalability:** Future features like AI or multiplayer can be integrated seamlessly by extending relevant modules.

User Interface Design

Components of the UI

1. Display Board

- Render the current state of the 3x3 board.
- Use symbols like X, O, and (space) for empty cells.
- Use lines and dividers (|, - - -) for clarity.

2. Player Input

- Prompt the player to input their move (row and column).
- Validate input to ensure it is within bounds.

3. Status Messages

- Display messages like:
 - o Whose turn it is.
 - o Invalid move prompts.
 - o Game outcomes (win, draw).

4. Restart Option

- Offer players a chance to restart or quit after a game ends.

Code Implementation

1. Displaying the Board

```
#include<stdio.h>

void displayBoard() {

    printf("\n");

    int i,j;

    for (i = 0; i < SIZE; i++) {

        for (j = 0; j < SIZE; j++) {

            printf(" %c ", board[i][j]);

            if (j < SIZE - 1) printf("|");

        }

    }
```

```

printf("\n");

if (i < SIZE - 1) {

    for (j = 0; j < SIZE; j++) {

        printf("---");

        if (j < SIZE - 1) printf("|");

    }

    printf("\n");

}

}

printf("\n");
}

```

2. Handling Player Input

```

void makeMove() {

    int row, col;

    int valid = 0;

    while (!valid) {

        printf("Player %c, enter your move (row and column: 1-3): ", current_player);

        scanf("%d %d", &row, &col);

        // Adjust for 0-based indexing

        row--;

        col--;

        if (isValidMove(row, col)) {

            board[row][col] = current_player;

```

```

        valid = 1;

    } else {

        printf("Invalid move. Try again.\n");

    }

}

}

```

3. Showing Status Messages

```

initializeBoard();

current_player = 'X';

printf("Welcome to Tic Tac Toe!\n");
printf("Player 1: X, Player 2: O\n\n");

while (!winner && !draw) {

    displayBoard();

    makeMove();

    winner = isWinner();

    draw = isDraw();

    current_player = (current_player == 'X') ? 'O' : 'X'; // Switch player

}

displayBoard();

if (winner) {

    printf("Player %c wins!\n", (current_player == 'X') ? 'O' : 'X');

} else if (draw) {

```

```
        printf("It's a draw!\n");
    }

    return 0;
}
```

4. Complete Integration Example

```
#include <stdio.h>

#define SIZE 3

char board[SIZE][SIZE];
char current_player;

void initializeBoard();
void displayBoard();
int isWinner();
int isDraw();
void makeMove();
int isValidMove(int row, int col);

int main() {
    int winner = 0, draw = 0;

    initializeBoard();
    current_player = 'X';

    printf("Welcome to Tic Tac Toe!\n");
    printf("Player 1: X, Player 2: O\n\n");
```



```

while (!winner && !draw) {

    displayBoard();

    makeMove();

    winner = isWinner();

    draw = isDraw();

    current_player = (current_player == 'X') ? 'O' : 'X'; // Switch player
}

displayBoard();

if (winner) {

    printf("Player %c wins!\n", (current_player == 'X') ? 'O' : 'X');

} else if (draw) {

    printf("It's a draw!\n");

}

return 0;

}

void initializeBoard() {

    int i;

    int j;

    for ( i = 0; i < SIZE; i++) {

        for (j = 0; j < SIZE; j++) {

            board[i][j] = ' ';

        }

    }

}

```

```
}
```

```
void displayBoard() {  
    printf("\n");  
    int i,j;  
    for (i = 0; i < SIZE; i++) {  
        for (j = 0; j < SIZE; j++) {  
            printf(" %c ", board[i][j]);  
            if (j < SIZE - 1) printf("|");  
        }  
        printf("\n");  
        if (i < SIZE - 1) {  
            for (j = 0; j < SIZE; j++) {  
                printf(" ---");  
                if (j < SIZE - 1) printf("|");  
            }  
            printf("\n");  
        }  
    }  
    printf("\n");  
}
```

```
int isWinner() {  
    // Check rows and columns  
    int i,j;  
    for ( i = 0; i < SIZE; i++) {  
        if ((board[i][0] == board[i][1] && board[i][1] == board[i][2] && board[i][0] != ' ') ||  
            (board[0][i] == board[1][i] && board[1][i] == board[2][i] && board[0][i] != ' ')) {
```

```

        return 1;
    }
}

if ((board[0][0] == board[1][1] && board[1][1] == board[2][2] && board[0][0] != ' ') ||
    (board[0][2] == board[1][1] && board[1][1] == board[2][0] && board[0][2] != ' ')) {
    return 1;
}

return 0;
}

int isDraw() {
    int i,j;
    for (i = 0; i < SIZE; i++) {
        for ( j = 0; j < SIZE; j++) {
            if (board[i][j] == ' ') {
                return 0;
            }
        }
    }
    return 1;
}

void makeMove() {
    int row, col;
    int valid = 0;

```

```

while (!valid) {

    printf("Player %c, enter your move (row and column: 1-3): ", current_player);

    scanf("%d %d", &row, &col);


    row--;

    col--;


    if (isValidMove(row, col)) {

        board[row][col] = current_player;

        valid = 1;

    } else {

        printf("Invalid move. Try again.\n");

    }

}

int isValidMove(int row, int col) {

    return (row >= 0 && row < SIZE && col >= 0 && col < SIZE && board[row][col] == ' ');

}

```

Features of this UI Design

1. **Text-Based Clarity:**
 - o The game board is displayed neatly with separators.
2. **Interactive Gameplay:**
 - o Players are prompted for their moves in a loop until the game ends.
3. **Error Handling:**
 - o Invalid moves are detected and players are asked to try again.
4. **Game End Handling:**
 - o Displays the winner or draw message and offers a restart option.
5. **Extensibility:**
 - o This design can be enhanced with features like AI or custom player names.

Testing

Testing Strategy

1. **Unit Tests:** Test individual functions, such as:
 - o `isValidMove`
 - o `checkWinner`
 - o Board initialization
2. **Integration Tests:** Simulate a sequence of actions and verify the output, such as:
 - o A complete game flow where one player wins.
 - o A game ending in a draw.
3. **Automation:** Use assertions (`assert` from `<assert.h>`) to automate checking expected results.

Key Functionalities to Test

1. **Display of the Board**
 - o Ensure the board is displayed correctly.
 - o Verify updates after each move.
2. **Player Input**
 - o Handle valid and invalid moves.
 - o Handle repeated moves on the same cell.
 - o Switch between players correctly.
3. **Win Conditions**
 - o Horizontal, vertical, and diagonal wins are detected properly.
 - o Both players can win under the correct conditions.
4. **Draw Condition**
 - o The game recognizes a draw when all cells are filled and there's no winner.
5. **Restart or Exit**
 - o Option to restart or exit the game works as intended.

Testing the Program

Automated Testing (Optional)

- Write test scripts to simulate user input.
- Use tools like `expect` (for Unix systems) to automate input/output testing.

Manual Testing

1. Run the program.
2. Test various scenarios:
 - o Sequential moves by two players.
 - o Winning conditions (horizontal, vertical, diagonal).
 - o Input validation (invalid numbers, duplicates).
 - o Edge cases (e.g., all cells filled without a winner).
3. Test replaying or exiting functionality.

Testing Technique and Testing Strategies Used

Testing a Tic Tac Toe program involves applying specific **testing techniques** and **testing strategies** to ensure it functions correctly and is free of bugs. Here's an outline:

1. Testing Techniques

A. Black-Box Testing

- Focuses on inputs and expected outputs without looking at the code.
- Examples:
 - o Test winning conditions by providing moves that form rows, columns, or diagonals.
 - o Test invalid inputs like non-numeric values or numbers outside the valid range (1-9).
 - o Test draw conditions by filling the board completely without a winner.

B. White-Box Testing

- Involves examining the internal logic of the code.
- Examples:
 - o Ensure every `if` condition is tested (e.g., win checks for rows, columns, diagonals).
 - o Test edge cases in functions like `checkWin` and `checkDraw`.
 - o Ensure all branches in `makeMove` handle inputs correctly (valid, invalid, duplicate).

C. Boundary Value Testing

- Tests edge conditions for inputs.
- Examples:
 - o Input 1 (lowest valid move).
 - o Input 9 (highest valid move).
 - o Input 0 or 10 (outside valid range).

D. Equivalence Partitioning

- Divides input data into partitions that are expected to behave similarly.
- Examples:
 - o Valid moves (1-9) vs. invalid moves (non-numeric, duplicates, numbers <1 or >9).
 - o Full board states leading to win, draw, or no result.

E. Error Guessing

- Relies on intuition to guess potential bugs.
- Examples:
 - o Repeated moves on the same cell.
 - o Rapid inputs causing unexpected behavior.
 - o Inputs that simulate malicious usage (e.g., injecting invalid data).

2. Testing Strategies

A. Unit Testing

- Test individual components or functions in isolation.
- Example:
 - o Test the `checkWin()` function with different board configurations.
 - o Test `initializeBoard()` to ensure the board resets correctly.

B. Integration Testing

- Test interactions between components.
- Example:
 - o Ensure `makeMove` updates the board, switches players, and interacts with `checkWin`.

C. System Testing

- Test the entire program as a whole.
- Example:
 - o Run full game sessions with various scenarios (e.g., Player X wins, Player O wins, draw).

D. Regression Testing

- Ensure that new changes or bug fixes don't break existing functionality.
- Example:
 - o After modifying `checkWin`, retest all win conditions.

E. Exploratory Testing

- Test the program without a predefined plan to discover unforeseen issues.
- Example:
 - o Input unexpected data like strings or special characters.
 - o Test how the program handles unusually fast user inputs.

F. Stress Testing

- Push the program beyond normal usage to check its robustness.
- Example:
 - o Enter invalid data repeatedly or provide extremely large input values.

G. User Acceptance Testing (UAT)

- Simulate real-world scenarios to check usability and satisfaction.
- Example:
 - o Play a few games to evaluate the user interface and overall experience.

3. Example Test Cases

Here are some sample test cases for manual or automated testing:

Test Case	Input	Expected Result
Valid Move	Player X inputs 5	5 replaced with X; Player O's turn.
Invalid Move (Out of Range)	Player X inputs 10	Error message shown; Player X retries.
Win Condition	Moves: 1, 4, 2, 5, 3	Player X wins with a horizontal line.
Draw Condition	Fill board with no winner	Program declares a draw.
Repeated Move	Player X inputs 3 twice	Error message shown; Player X retries.
Non-Numeric Input	Player X inputs a	Error message shown; Player X retries.
Restart Game	Select restart after game ends	Board resets; Player X starts.

4. Tools for Testing

- **Manual Testing:** Play and verify outputs based on scenarios.
- **Automated Testing:**
 - o Use frameworks like CUnit (for unit testing in C).
 - o Use scripting tools (e.g., expect or Python scripts) to automate input/output testing.

System Security Measure

Implementing system security measures for a Tic Tac Toe program may seem unnecessary for a simple application. However, if the program is used in a broader context (e.g., as part of a web-based system or on a shared platform), certain security considerations are important. Below are some relevant security measures:

1. Input Validation and Sanitization

- **Validate User Input:**
 - o Ensure only valid inputs (e.g., numbers between 1-9) are accepted.
 - o Prevent unexpected inputs like special characters, strings, or malicious input.
- **Sanitize Inputs:**
 - o Strip or reject harmful inputs to avoid injection attacks, especially if the program interacts with files or databases.

2. Memory Management

- **Avoid Buffer Overflows:**
 - o Ensure arrays and buffers used for input are appropriately sized.
 - o Use safe functions like `fgetc`s for input instead of `getc`s (which is unsafe).
- **Free Allocated Memory:**
 - o If dynamic memory allocation is used, ensure it is freed properly to avoid memory leaks.

3. Error Handling

- Handle unexpected behavior gracefully.
- Display informative but non-technical error messages to avoid revealing system details.
- Use exit codes for debugging without crashing the program.

4. Access Control

- If the game is multi-user or online:
 - o Authenticate users to prevent unauthorized access.
 - o Limit user privileges to necessary actions (e.g., playing the game).

5. Logging and Monitoring

- Log user actions to detect unusual activity (e.g., repeated invalid inputs, tampering attempts).
- Avoid storing sensitive information like passwords in logs.

6. File and Data Security

- If the game stores data (e.g., scores, settings):
 - Encrypt sensitive data to prevent unauthorized access.
 - Use secure file handling to avoid vulnerabilities like path traversal attacks.

7. Network Security (If Online)

- **Use HTTPS:** Encrypt data transmitted between the server and clients.
- **Rate Limiting:** Prevent abuse by limiting the number of requests from a single IP in a time frame.
- **Authentication Tokens:** Use tokens to validate user sessions and prevent tampering.

8. Protect Against Denial of Service (DoS)

- If hosted online, implement measures to limit excessive resource usage:
 - Time out inactive sessions.
 - Limit the size and frequency of user inputs.

9. Code Security

- **Use Secure Coding Practices:**
 - Avoid hardcoding sensitive data (e.g., credentials or keys).
 - Use consistent coding standards to minimize bugs.
- **Static Code Analysis:**
 - Use tools like cppcheck to detect vulnerabilities during development.

10. Update and Patch Management

- Regularly update the program to fix vulnerabilities.
- Apply security patches to third-party libraries or dependencies.

11. User Awareness

- Educate users on safe usage practices, such as avoiding sharing login credentials (if applicable).

Security Checklist

Here's a quick checklist to evaluate the security of the system:

Measure	Implemented?
Input Validation	☒ / ☒
Memory Management	☒ / ☒
Error Handling	☒ / ☒
Authentication	☒ / ☒
Data Encryption	☒ / ☒
Secure File Handling	☒ / ☒
HTTPS for Communication	☒ / ☒
Rate Limiting	☒ / ☒
Static Code Analysis	☒ / ☒
Patch Management	☒ / ☒

Future Scope and Further Enhancement On This Project

Enhancing a Tic Tac Toe project involves exploring features, platforms, and technologies that go beyond the basic functionality, adding depth and interactivity. Here are ideas for **future scope** and **enhancements**:

1. User Experience Enhancements

- **Dynamic UI/UX:**
 - o Replace the command-line interface with a graphical user interface (GUI) using libraries like **SDL (C)**, **Qt**, or **GTK**.
 - o Add animations and sound effects for player moves, wins, and draws.
- **Custom Themes:**
 - o Allow players to choose board designs, colors, or themes.
- **Game Modes:**
 - o Add difficulty levels, such as *easy*, *medium*, and *hard*.
 - o Introduce timed moves to make the game more challenging.

2. Multiplayer Features

- **Local Multiplayer:**
 - o Enhance the program to support two players on the same device with better input mechanisms.
- **Online Multiplayer:**
 - o Allow players to compete over the internet using **sockets** for communication.
 - o Use platforms like **WebSocket** or **REST APIs** for real-time gameplay.
- **Leaderboards:**
 - o Implement score tracking and display global or local leaderboards.

3. AI Improvements

- **Advanced AI:**
 - o Implement a more challenging AI using:
 - **Minimax Algorithm:** Ensure the AI plays optimally.
 - **Machine Learning:** Train an AI to improve based on previous games.
- **Adaptive AI:**
 - o Create an AI that learns a player's strategies and adjusts its moves accordingly.

4. Cross-Platform Deployment

- **Web-Based Game:**
 - Develop a browser-based version using **HTML**, **CSS**, and **JavaScript** or frameworks like **React** or **Angular**.
- **Mobile App:**
 - Create a mobile version using platforms like **Flutter**, **React Native**, or native development tools (e.g., Android Studio for Android, Xcode for iOS).
- **Console/Embedded Systems:**
 - Port the game to gaming consoles or embedded systems for a unique gaming experience.

5. Educational Features

- **Learning Mode:**
 - Include tutorials to teach new players strategies and tips for improving gameplay.
- **Analysis Mode:**
 - Allow players to replay games and analyze moves with suggestions for improvement.

6. Gamification

- **Achievements and Rewards:**
 - Introduce badges, trophies, and rewards for milestones (e.g., winning streaks, challenging AI wins).
- **Progression System:**
 - Add levels or ranks that players can achieve by accumulating points.

7. Integration with Emerging Technologies

- **Voice Commands:**
 - Enable players to control the game using voice input with tools like **Google Speech API**.
- **Augmented Reality (AR):**
 - Implement the game in AR for platforms like **Unity** with AR support.
- **Blockchain Integration:**
 - Use blockchain to create a transparent leaderboard or enable NFT rewards for winners.

8. Advanced Features

- **Custom Game Rules:**
 - o Allow players to tweak game rules, such as board size (e.g., 4x4 or 5x5) or alternate win conditions.
- **Save and Resume:**
 - o Implement a feature to save game progress and resume later.
- **Spectator Mode:**
 - o Allow others to watch ongoing multiplayer games.

9. System Integration

- **Cloud Integration:**
 - o Store game data, user profiles, and leaderboards on the cloud for seamless access across devices.
- **IoT Integration:**
 - o Create a physical Tic Tac Toe board that syncs with the digital game.

10. Analytical Features

- **Game Analytics:**
 - o Provide statistics for players, such as win/loss ratio, average moves per game, and common mistakes.
- **AI Analytics:**
 - o Offer insights into AI behavior and decisions during the game.

11. Community Engagement

- **User-Generated Content:**
 - o Let users create and share custom boards or AI strategies.
- **Tournaments:**
 - o Organize online tournaments with matchmaking for competitive play.

12. Technical Improvements

- **Code Optimization:**
 - o Refactor code for better performance and maintainability.
- **Modular Design:**
 - o Make the program modular to facilitate future updates and feature additions.
- **Unit Testing and CI/CD:**
 - o Integrate unit testing and a CI/CD pipeline for robust development practices.

Appendices

1. Coding (along with comments and proper indentation)

```
#include <stdio.h>

#define SIZE 3

char board[SIZE][SIZE];
char current_player;

// Function prototypes
void initializeBoard();
void displayBoard();
int isWinner();
int isDraw();
void makeMove();
int isValidMove(int row, int col);

int main() {
    int winner = 0, draw = 0;

    initializeBoard();
    current_player = 'X';

    printf("Welcome to Tic Tac Toe!\n");
    printf("Player 1: X, Player 2: O\n\n");

    while (!winner && !draw) {
        displayBoard();
        makeMove();
```

```

        winner = isWinner();

        draw = isDraw();

        current_player = (current_player == 'X') ? 'O' : 'X'; // Switch player
    }

    displayBoard();

    if (winner) {
        printf("Player %c wins!\n", (current_player == 'X') ? 'O' : 'X');
    } else if (draw) {
        printf("It's a draw!\n");
    }

    return 0;
}

// Initialize the board with empty spaces
void initializeBoard() {
    int i;
    int j;
    for ( i = 0; i < SIZE; i++) {
        for (j = 0; j < SIZE; j++) {
            board[i][j] = ' ';
        }
    }
}

// Display the current state of the board

```

```

void displayBoard() {

    printf("\n");

    int i,j;

    for (i = 0; i < SIZE; i++) {

        for (j = 0; j < SIZE; j++) {

            printf(" %c ", board[i][j]);

            if (j < SIZE - 1) printf("|");

        }

        printf("\n");

        if (i < SIZE - 1) {

            for (j = 0; j < SIZE; j++) {

                printf(" ---");

                if (j < SIZE - 1) printf("|");

            }

            printf("\n");

        }

    }

    printf("\n");
}

```

// Check if there's a winner

```

int isWinner() {

    // Check rows and columns

    int i,j;

    for ( i = 0; i < SIZE; i++) {

        if ((board[i][0] == board[i][1] && board[i][1] == board[i][2] && board[i][0] != ' ') ||

            (board[0][i] == board[1][i] && board[1][i] == board[2][i] && board[0][i] != ' ')) {

            return 1;

```

```

    }
}

// Check diagonals
if ((board[0][0] == board[1][1] && board[1][1] == board[2][2] && board[0][0] != ' ') ||
    (board[0][2] == board[1][1] && board[1][1] == board[2][0] && board[0][2] != ' ')) {
    return 1;
}

return 0;
}

// Check if the game is a draw
int isDraw() {
    int i,j;
    for (i = 0; i < SIZE; i++) {
        for ( j = 0; j < SIZE; j++) {
            if (board[i][j] == ' ') {
                return 0;
            }
        }
    }
    return 1;
}

// Allow the current player to make a move
void makeMove() {
    int row, col;

```

```

int valid = 0;

while (!valid) {

    printf("Player %c, enter your move (row and column: 1-3): ", current_player);

    scanf("%d %d", &row, &col);

    // Adjust for 0-based indexing

    row--;

    col--;

    if (isValidMove(row, col)) {

        board[row][col] = current_player;

        valid = 1;

    } else {

        printf("Invalid move. Try again.\n");

    }

}

// Check if a move is valid

int isValidMove(int row, int col) {

    return (row >= 0 && row < SIZE && col >= 0 && col < SIZE && board[row][col] == ' ');

}

```

```
C:\Users\lenovo\Desktop\tictactoe.c - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Classes Debug [*] tictactoe.c

1  #include <stdio.h>
2  #define SIZE 3
3
4  char board[SIZE][SIZE];
5  char current_player;
6
7  // Function prototypes
8  void initializeBoard();
9  void displayBoard();
10 int isWinner();
11 int isDraw();
12 void makeMove();
13 int isValidMove(int row, int col);
14
15 int main() {
16     int winner = 0, draw = 0;
17
18     initializeBoard();
19     current_player = 'X';
20
21     printf("Welcome to Tic Tac Toe!\n");
22     printf("Player 1: X, Player 2: O\n\n");
23 }
```

```
C:\Users\lenovo\Desktop\tictactoe.c - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Classes Debug [*] tictactoe.c

23
24 while (!winner && !draw) {
25     displayBoard();
26     makeMove();
27     winner = isWinner();
28     draw = isDraw();
29     current_player = (current_player == 'X') ? 'O' : 'X'; // Switch player
30 }
31
32 displayBoard();
33
34 if (winner) {
35     printf("Player %c wins!\n", (current_player == 'X') ? 'O' : 'X');
36 } else if (draw) {
37     printf("It's a draw!\n");
38 }
39
40 return 0;
41 }
```

```
C:\Users\lenovo\Desktop\tictactoe.c - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Classes Debug [*] tictactoe.c

43 // Initialize the board with empty spaces
44 void initializeBoard() {
45     int i;
46     int j;
47     for ( i = 0; i < SIZE; i++) {
48         for (j = 0; j < SIZE; j++) {
49             board[i][j] = ' ';
50         }
51     }
52 }
53
54 // Display the current state of the board
55 void displayBoard() {
56     printf("\n");
57     int i,j;
58     for (i = 0; i < SIZE; i++) {
59         for (j = 0; j < SIZE; j++) {
60             printf(" %c ", board[i][j]);
61             if (j < SIZE - 1) printf("|");
62         }
63         printf("\n");
64         if (i < SIZE - 1) {
65             for (j = 0; j < SIZE; j++) {
66                 printf("---");
67                 if (j < SIZE - 1) printf("|");
68             }
69             printf("\n");

```

```
C:\Users\lenovo\Desktop\tictactoe.c - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Classes Debug [*] tictactoe.c

70     }
71 }
72 printf("\n");
73 }
74
75 // Check if there's a winner
76 int isWinner() {
77     // Check rows and columns
78     int i,j;
79     for ( i = 0; i < SIZE; i++) {
80         if ((board[i][0] == board[i][1] && board[i][1] == board[i][2] && board[i][0] != ' ') ||
81             (board[0][i] == board[1][i] && board[1][i] == board[2][i] && board[0][i] != ' ')) {
82             return 1;
83         }
84     }
85
86     // Check diagonals
87     if ((board[0][0] == board[1][1] && board[1][1] == board[2][2] && board[0][0] != ' ') ||
88         (board[0][2] == board[1][1] && board[1][1] == board[2][0] && board[0][2] != ' ')) {
89         return 1;
90     }
91
92     return 0;
93 }
```

```
C:\Users\lenovo\Desktop\tictactoe.c - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Classes Debug ["tictactoe.c"]

94
95 // Check if the game is a draw
96 int isDraw() {
97     int i,j;
98     for (i = 0; i < SIZE; i++) {
99         for (j = 0; j < SIZE; j++) {
100             if (board[i][j] == ' ') {
101                 return 0;
102             }
103         }
104     }
105     return 1;
106 }
107
108 // Allow the current player to make a move
109 void makeMove() {
110     int row, col;
111     int valid = 0;
112
113     while (!valid) {
114         printf("Player %c, enter your move (row and column: 1-3): ", current_player);
115         scanf("%d %d", &row, &col);
116     }
```

```
C:\Users\lenovo\Desktop\tictactoe.c - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Classes Debug ["tictactoe.c"]

107
108 // Allow the current player to make a move
109 void makeMove() {
110     int row, col;
111     int valid = 0;
112
113     while (!valid) {
114         printf("Player %c, enter your move (row and column: 1-3): ", current_player);
115         scanf("%d %d", &row, &col);
116
117         // Adjust for 0-based indexing
118         row--;
119         col--;
120
121         if (isValidMove(row, col)) {
122             board[row][col] = current_player;
123             valid = 1;
124         } else {
125             printf("Invalid move. Try again.\n");
126         }
127     }
128 }
129
130 // Check if a move is valid
131 int isValidMove(int row, int col) {
132     return (row >= 0 && row < SIZE && col >= 0 && col < SIZE && board[row][col] == ' ');
133 }
134

Compiler Resources Compile Log Debug Find Results
```


Output Of This Code

If the player is win then the given output is:

```
C:\Users\lenovo\Desktop\tict: X + v
Welcome to Tic Tac Toe!
Player 1: X, Player 2: O

| | |
---|---|---
| | |
---|---|---
| | |

Player X, enter your move (row and column: 1-3): |
```

```
Player X, enter your move (row and column: 1-3): 1
2
| X |
---|---|---
| | |
---|---|---
| | |

Player O, enter your move (row and column: 1-3): 1
3
| X | O
---|---|---
| | |
---|---|---
| | |

Player X, enter your move (row and column: 1-3): 2
2
| X | O
---|---|---
| X |
---|---|---
| | |
```

```

---|---|---
   | X |   |
---|---|---
   |   |   |

Player 0, enter your move (row and column: 1-3): 2
3

   | X | 0 |
---|---|---
   | X | 0 |
---|---|---
   |   |   |

Player X, enter your move (row and column: 1-3): 3
2

   | X | 0 |
---|---|---
   | X | 0 |
---|---|---
   | X |   |

Player X wins!

-----
Process exited after 47.92 seconds with return value 0
Press any key to continue . . . |

```

If the match is draw then the given output is:

```
C:\Users\lenovo\Desktop\tict: X + v

---|---|---
---|---|---
---|---|---

Player X, enter your move (row and column: 1-3): 2
2

---|---|---
---| X |---
---|---|---

Player 0, enter your move (row and column: 1-3): 2
1

---|---|---
0 | X |---
---|---|---

Player X, enter your move (row and column: 1-3): 3
1

---|---|---
0 | X |---
X |---|---
```

```
C:\Users\lenovo\Desktop\tict: X + v

Player 0, enter your move (row and column: 1-3): 1
3

  |  |  | 0
---|---|---
 0 | X | 
---|---|---
 X |  | 
---|---|---

Player X, enter your move (row and column: 1-3): 1
1

 X |  | 0
---|---|---
 0 | X | 
---|---|---
 X |  | 
---|---|---

Player 0, enter your move (row and column: 1-3): 3
3

 X |  | 0
---|---|---
 0 | X | 
---|---|---
 X |  | 0
---|---|---

Player X, enter your move (row and column: 1-3): 2
3

 X |  | 0
---|---|---
 0 | X | X
---|---|---
 X |  | 0
---|---|---
```

```
Player 0, enter your move (row and column: 1-3): 3
2
```

```

X |  | 0
---|---|---
0 | X | X
---|---|---
X | 0 | 0
```

```
Player X, enter your move (row and column: 1-3): 1
2
```

```

X | X | 0
---|---|---
0 | X | X
---|---|---
X | 0 | 0
```

```
It's a draw!
```

```
-----
Process exited after 59.57 seconds with return value 0
Press any key to continue . . . |
```

Bibliography

Online Resources:-

- Game Development Stack Exchange.
- Rogers, Ian. History of Pac-Man: The Legendary Arcade Game. Retro Gaming Press, 2018

Websites:-

- Game Development Resources
- GitHub.